Министерство образования Республики Беларусь

Учреждения образования

«Брестский государственный технический университет

Факультет электронно-информационных систем

Кафедра интеллектуальных информационных технологий

Отчет по лабораторной работе №6

Специальность ПО-12

Выполнил
К. В. Ковалюк,
студент группы ПО-12

Проверил
А. Д. Кулик,
преп. стаж. кафедры ИИТ,
«___» _____2026 г.

Брест 2026

Цель работы: Освоить приемы тестирования кода на примере использования пакета pytest.

Вариант: №7

Реализовать простой класс

Напишите метод String substringBetween(String str, String open, String close) выделяющий подстроку относительно открывающей и закрывающей строки.

Спецификация метода:

substringBetween (None , None, None) = TypeError
substringBetween (None, * , *) = None
substringBetween (* , None, *) = None
substringBetween (* , * , None) = None
substringBetween ("", "", "") = ""
substringBetween ("", "", "]") = None
substringBetween ("", "[", "]") = None
substringBetween (" yabcz ", "", "") = ""
substringBetween (" yabcz ", "y", "z") = " abc"
substringBetween (" yabczyabcz ", "y", "z") = " abc "
substringBetween ("wx[b]yz", "[", "]") = "b"

Задание 1: Написание тестов для мини-библиотеки покупок (shopping.py)

1. Создайте файл test_cart.py. Реализуйте следующие тесты:
   - Проверка добавления товара: после add_item("Apple", 10.0) в корзине должен быть один элемент.
   - Проверка выброса ошибки при отрицательной цене.
   - Проверка вычисления общей стоимости (total()).
2. Протестируйте метод apply_discount с разными значениями скидки:
   - 0% - цена остаётся прежней
   - 50% - цена уменьшается вдвое
   - 100% - цена становится ноль
   - < 0% и > 100% - должно выбрасываться исключение

   Используйте @pytest.mark.parametrize
3. Создайте фикстуру empty_cart, которая возвращает пустой экземпляр Cart@pytest.fixture

   def empty_cart(): return Cart()

   Используйте эту фикстуру в тестах, где нужно создать новую корзину.
4. Допустим, у нас есть функция, которая логирует покупку в удалённую систему:

   import requests
   def log_purchase(item):
   requests.post("https://example.com/log", json=item)
   - Замокайте requests.post, чтобы не было реального HTTP-запроса
   - Убедитесь, что он вызывается с корректными данными
5. Добавьте поддержку купонов:

   def apply_coupon(cart, coupon_code):
   coupons = {"SAVE10": 10, "HALF": 50}
   if coupon_code in coupons:
   cart.apply_discount(coupons[coupon_code])
   else:
   raise ValueError("Invalid coupon")
   - Напишите тесты на apply_coupon
   - Замокайте словарь coupons с помощью monkeypatch или patch.dict

Задание 2: Напишите тесты к реализованным функциям из лабораторной работы №1. Проверьте тривиальные и граничные случае, а также варианты, когда может возникнуть

исключительная ситуация. Если при реализации не использовались отдельные функции, необходимо провести рефакторинг кода.

Задание 3: Написать тесты к методу, а затем реализовать сам метод по заданной спецификации.

Выполнение:

Код программы

```
import pytest
from unittest.mock import patch, MagicMock
import requests


# ==================== ЗАДАНИЕ 1: Мини-библиотека покупок ====================

class Cart:
    def __init__(self):
        self.items = []

    def add_item(self, name, price):
        if price < 0:
            raise ValueError("Price cannot be negative")
        self.items.append({"name": name, "price": price})

    def total(self):
        return sum(item["price"] for item in self.items)

    def apply_discount(self, discount_percent):
        if discount_percent < 0 or discount_percent > 100:
            raise ValueError("Discount must be between 0 and 100")

        multiplier = (100 - discount_percent) / 100
        for item in self.items:
            item["price"] = round(item["price"] * multiplier, 2)

    def clear(self):
        self.items = []


def log_purchase(item):
    response = requests.post("https://example.com/log", json=item)
    return response


COUPONS = {"SAVE10": 10, "HALF": 50}


def apply_coupon(cart, coupon_code):
    if coupon_code in COUPONS:
        cart.apply_discount(COUPONS[coupon_code])
    else:
        raise ValueError("Invalid coupon")


# ==================== ЗАДАНИЕ 2: CharSet из ЛР1 ====================

class CharSet:
    def __init__(self, max_size=10, initial_chars=None):
        self._max_size = max_size
        self._items = []
        if initial_chars:
            for char in initial_chars:
                self.add(char)

    def get_items(self):
```

```
        return self._items.copy()

    def get_max_size(self):
        return self._max_size

    def add(self, char):
        if len(char) != 1:
            print(f"Ошибка: '{char}' не является одиночным символом.")
            return False
        if char in self._items:
            print(f"Элемент '{char}' уже существует в множестве.")
            return False
        if len(self._items) >= self._max_size:
            print(f"Ошибка: Достигнута максимальная мощность множества ({self._max_size}). Элемент
'{char}' не добавлен.")
            return False
        self._items.append(char)
        print(f"Элемент '{char}' успешно добавлен.")
        return True

    def remove(self, char):
        if char in self._items:
            self._items.remove(char)
            print(f"Элемент '{char}' удален.")
            return True
        print(f"Ошибка: Элемент '{char}' не найден.")
        return False

    def contains(self, char):
        return char in self._items

    def union(self, other):
        combined_chars = list(set(self.get_items() + other.get_items()))
        new_set = CharSet(max(self.get_max_size(), other.get_max_size()))
        for char in combined_chars:
            new_set.add(char)
        return new_set

    def intersection(self, other):
        intersected_chars = [char for char in self.get_items() if char in other.get_items()]
        new_set = CharSet(min(self.get_max_size(), other.get_max_size()))
        for char in intersected_chars:
            new_set.add(char)
        return new_set

    def difference(self, other):
        diff_chars = [char for char in self.get_items() if char not in other.get_items()]
        new_set = CharSet(self.get_max_size())
        for char in diff_chars:
            new_set.add(char)
        return new_set

    def display(self):
        print(f"Множество (мощность {len(self._items)}/{self._max_size}): {self._items}")

    def __str__(self):
        return f"CharSet(capacity={self._max_size}, items={self._items})"

    def __eq__(self, other):
        if not isinstance(other, CharSet):
            return False
        return self._max_size == other._max_size and set(self._items) == set(other._items)


# ==================== ЗАДАНИЕ 3: substringBetween ====================
```

```
def substring_between(str_input, open_str, close_str):
    if str_input is None or open_str is None or close_str is None:
        raise TypeError("None values are not allowed")

    if open_str == "" and close_str == "":
        return ""

    if open_str == "":
        start = 0
    else:
        start = str_input.find(open_str)
        if start == -1:
            return None
        start += len(open_str)

    if close_str == "":
        end = len(str_input)
    else:
        end = str_input.find(close_str, start)
        if end == -1:
            return None

    return str_input[start:end]


# ==================== ТЕСТЫ ДЛЯ ЗАДАНИЯ 1 ====================

@pytest.fixture
def empty_cart():
    return Cart()


def test_add_item(empty_cart):
    empty_cart.add_item("Apple", 10.0)
    assert len(empty_cart.items) == 1
    assert empty_cart.items[0]["name"] == "Apple"
    assert empty_cart.items[0]["price"] == 10.0


def test_add_item_negative_price(empty_cart):
    with pytest.raises(ValueError, match="Price cannot be negative"):
        empty_cart.add_item("Apple", -10.0)


def test_total(empty_cart):
    empty_cart.add_item("Apple", 10.0)
    empty_cart.add_item("Banana", 15.0)
    assert empty_cart.total() == 25.0


@pytest.mark.parametrize("discount,expected_prices", [
    (0, [10.0, 15.0]),
    (50, [5.0, 7.5]),
    (100, [0.0, 0.0]),
])
def test_apply_discount_valid(empty_cart, discount, expected_prices):
    empty_cart.add_item("Apple", 10.0)
    empty_cart.add_item("Banana", 15.0)
    empty_cart.apply_discount(discount)

    assert empty_cart.items[0]["price"] == expected_prices[0]
    assert empty_cart.items[1]["price"] == expected_prices[1]


@pytest.mark.parametrize("discount", [-10, 110, -50, 150])
def test_apply_discount_invalid(empty_cart, discount):
```

```
    empty_cart.add_item("Apple", 10.0)
    with pytest.raises(ValueError, match="Discount must be between 0 and 100"):
        empty_cart.apply_discount(discount)


@patch("requests.post")
def test_log_purchase(mock_post):
    mock_response = MagicMock()
    mock_post.return_value = mock_response

    item = {"name": "Apple", "price": 10.0}
    log_purchase(item)

    mock_post.assert_called_once_with(
        "https://example.com/log",
        json=item
    )


def test_apply_coupon_valid(empty_cart):
    empty_cart.add_item("Apple", 100.0)
    apply_coupon(empty_cart, "SAVE10")
    assert empty_cart.items[0]["price"] == 90.0

    empty_cart.clear()
    empty_cart.add_item("Banana", 100.0)
    apply_coupon(empty_cart, "HALF")
    assert empty_cart.items[0]["price"] == 50.0


def test_apply_coupon_invalid(empty_cart):
    empty_cart.add_item("Apple", 100.0)
    with pytest.raises(ValueError, match="Invalid coupon"):
        apply_coupon(empty_cart, "INVALID")


# ==================== ТЕСТЫ ДЛЯ ЗАДАНИЯ 2 ====================

def test_char_set_add_valid_char():
    cs = CharSet(5)
    result = cs.add("a")
    assert result == True
    assert cs.contains("a") == True
    assert len(cs.get_items()) == 1


def test_char_set_add_existing_char():
    cs = CharSet(5, ["a"])
    result = cs.add("a")
    assert result == False
    assert len(cs.get_items()) == 1


def test_char_set_add_to_full_set():
    cs = CharSet(2, ["a", "b"])
    result = cs.add("c")
    assert result == False
    assert len(cs.get_items()) == 2


def test_char_set_add_invalid_char():
    cs = CharSet(5)
    result = cs.add("abc")
    assert result == False
    assert len(cs.get_items()) == 0
```

```
def test_char_set_remove_existing_char():
    cs = CharSet(5, ["a", "b"])
    result = cs.remove("a")
    assert result == True
    assert cs.contains("a") == False
    assert len(cs.get_items()) == 1


def test_char_set_remove_non_existing_char():
    cs = CharSet(5, ["a", "b"])
    result = cs.remove("c")
    assert result == False
    assert len(cs.get_items()) == 2


def test_char_set_contains():
    cs = CharSet(5, ["a", "b"])
    assert cs.contains("a") == True
    assert cs.contains("c") == False


def test_char_set_union():
    cs1 = CharSet(5, ["a", "b"])
    cs2 = CharSet(5, ["b", "c"])
    result = cs1.union(cs2)
    assert set(result.get_items()) == {"a", "b", "c"}
    assert result.get_max_size() == 5


def test_char_set_intersection():
    cs1 = CharSet(5, ["a", "b", "c"])
    cs2 = CharSet(5, ["b", "c", "d"])
    result = cs1.intersection(cs2)
    assert set(result.get_items()) == {"b", "c"}


def test_char_set_difference():
    cs1 = CharSet(5, ["a", "b", "c"])
    cs2 = CharSet(5, ["b", "c", "d"])
    result = cs1.difference(cs2)
    assert set(result.get_items()) == {"a"}


def test_char_set_empty_set():
    cs = CharSet()
    assert len(cs.get_items()) == 0
    assert cs.get_max_size() == 10


def test_char_set_initial_chars():
    cs = CharSet(5, ["a", "b", "c"])
    assert len(cs.get_items()) == 3
    assert cs.contains("a") == True
    assert cs.contains("b") == True
    assert cs.contains("c") == True


def test_char_set_max_size():
    cs = CharSet(3)
    cs.add("a")
    cs.add("b")
    cs.add("c")
    result = cs.add("d")
    assert result == False
    assert len(cs.get_items()) == 3
```

```
# ==================== ТЕСТЫ ДЛЯ ЗАДАНИЯ 3 ====================

def test_substring_between_none_none_none():
    with pytest.raises(TypeError, match="None values are not allowed"):
        substring_between(None, None, None)


def test_substring_between_none_any_any():
    with pytest.raises(TypeError, match="None values are not allowed"):
        substring_between(None, "y", "z")
    with pytest.raises(TypeError, match="None values are not allowed"):
        substring_between(None, "", "]")


def test_substring_between_any_none_any():
    with pytest.raises(TypeError, match="None values are not allowed"):
        substring_between("yabcz", None, "z")


def test_substring_between_any_any_none():
    with pytest.raises(TypeError, match="None values are not allowed"):
        substring_between("yabcz", "y", None)


def test_substring_between_empty_empty_empty():
    result = substring_between("", "", "")
    assert result == ""


def test_substring_between_empty_empty_close():
    result = substring_between("", "", "]")
    assert result is None


def test_substring_between_empty_open_close():
    result = substring_between("", "[", "]")
    assert result is None


def test_substring_between_with_spaces():
    result = substring_between(" yabcz ", "y", "z")
    assert result == "abc"


def test_substring_between_multiple_occurrences():
    result = substring_between(" yabczyabcz ", "y", "z")
    assert result == "abc"


def test_substring_between_with_brackets():
    result = substring_between("wx[b]yz", "[", "]")
    assert result == "b"


def test_substring_between_no_open():
    result = substring_between("abc", "x", "z")
    assert result is None


def test_substring_between_no_close():
    result = substring_between("abc", "a", "z")
    assert result is None
```

```
def test_substring_between_empty_open():
    result = substring_between("abc", "", "c")
    assert result == "ab"


def test_substring_between_empty_close():
    result = substring_between("abc", "a", "")
    assert result == "bc"


def test_substring_between_open_at_start():
    result = substring_between("[hello]world", "[", "]")
    assert result == "hello"


def test_substring_between_close_at_end():
    result = substring_between("hello[world]", "[", "]")
    assert result == "world"


if __name__ == "__main__":
    pytest.main([__file__, "-v", "--tb=short"])
```

Результат работы программы:

```
PS C:\DZ\3Kurs2Part\SPP\spp_po12\reports\Kovaluk_K_V\Lab6\src> python -c "import pytest; pytest.main(['Lab6.py', '-v'])"
============================================================================== test session starts ==============================================================================
platform win32 -- Python 3.13.2, pytest-7.4.3, pluggy-1.6.0 -- C:\Python313\python.exe
cachedir: .pytest_cache
rootdir: C:\DZ\3Kurs2Part\SPP\spp_po12\reports\Kovaluk_K_V\Lab6\src
plugins: anyio-3.7.1, mock-3.12.0
collected 42 items

Lab6.py::test_add_item                                                   PASSED [  2%]
Lab6.py::test_add_item_negative_price                                    PASSED [  4%]
Lab6.py::test_total                                                      PASSED [  7%]
Lab6.py::test_apply_discount_valid[0-expected_prices0]                   PASSED [  9%]
Lab6.py::test_apply_discount_valid[50-expected_prices1]                  PASSED [ 11%]
Lab6.py::test_apply_discount_valid[100-expected_prices2]                 PASSED [ 14%]
Lab6.py::test_apply_discount_invalid[-10]                                PASSED [ 16%]
Lab6.py::test_apply_discount_invalid[110]                                PASSED [ 19%]
Lab6.py::test_apply_discount_invalid[-50]                                PASSED [ 21%]
Lab6.py::test_apply_discount_invalid[150]                                PASSED [ 23%]
Lab6.py::test_log_purchase                                               PASSED [ 26%]
Lab6.py::test_apply_coupon_valid                                         PASSED [ 28%]
Lab6.py::test_apply_coupon_invalid                                       PASSED [ 30%]
Lab6.py::test_char_set_add_valid_char                                    PASSED [ 33%]
```

Lab6.py::test_char_set_add_existing_char PASSED [35%]
Lab6.py::test_char_set_add_to_full_set PASSED [38%]
Lab6.py::test_char_set_add_invalid_char PASSED [40%]
Lab6.py::test_char_set_remove_existing_char PASSED [42%]
Lab6.py::test_char_set_remove_non_existing_char PASSED [45%]
Lab6.py::test_char_set_contains PASSED [47%]
Lab6.py::test_char_set_union PASSED [50%]
Lab6.py::test_char_set_intersection PASSED [52%]
Lab6.py::test_char_set_difference PASSED [54%]
Lab6.py::test_char_set_empty_set PASSED [57%]
Lab6.py::test_char_set_initial_chars PASSED [59%]
Lab6.py::test_char_set_max_size PASSED [61%]
Lab6.py::test_substring_between_none_none_none PASSED [64%]
Lab6.py::test_substring_between_none_any_any PASSED [66%]
Lab6.py::test_substring_between_any_none_any PASSED [69%]
Lab6.py::test_substring_between_any_any_none PASSED [71%]
Lab6.py::test_substring_between_empty_empty_empty PASSED [73%]
Lab6.py::test_substring_between_empty_empty_close PASSED [76%]
Lab6.py::test_substring_between_empty_open_close PASSED [78%]
Lab6.py::test_substring_between_with_spaces PASSED [80%]
Lab6.py::test_substring_between_multiple_occurrences PASSED [83%]
Lab6.py::test_substring_between_with_brackets PASSED [85%]
Lab6.py::test_substring_between_no_open PASSED [88%]
Lab6.py::test_substring_between_no_close PASSED [90%]
Lab6.py::test_substring_between_empty_open PASSED [92%]
Lab6.py::test_substring_between_empty_close PASSED [95%]
Lab6.py::test_substring_between_open_at_start PASSED [97%]
Lab6.py::test_substring_between_close_at_end PASSED [100%]

========== 42 passed in 0.74s ======

Рисунок с результатом работы программы:

```
Lab6.py 2, U ×
C: > DZ > 3Kurs2Part > SPP > spp_po12 > reports > Kovaluk_K_V > Lab6 > src > Lab6.py > ...
392  def test_substring_between_empty_close():
393      result = substring_between("abc", "a", "")
394      assert result == "bc"
395
396
397  def test_substring_between_open_at_start():
398      result = substring_between("[hello]world", "[", "]")
399      assert result == "hello"
400
401
402  def test_substring_between_close_at_end():
403      result = substring_between("hello[world]", "[", "]")
404      assert result == "world"
405
406
407  if __name__ == "__main__":
408      pytest.main([__file__, "-v", "--tb=short"])
409
```

```
PROBLEMS 2   OUTPUT   DEBUG CONSOLE   TERMINAL   PORTS

PS C:\DZ\3Kurs2Part\SPP\spp_po12\reports\Kovaluk_K_V\Lab6\src> python -c "import pytest; pytest.main(['Lab6.py', '-v'])"
Lab6.py::test_substring_between_empty_empty_empty PASSED                                  [ 73%]
Lab6.py::test_substring_between_empty_empty_close PASSED                                  [ 76%]
Lab6.py::test_substring_between_empty_open_close PASSED                                   [ 78%]
Lab6.py::test_substring_between_with_spaces PASSED                                        [ 80%]
Lab6.py::test_substring_between_multiple_occurrences PASSED                               [ 83%]
Lab6.py::test_substring_between_with_brackets PASSED                                      [ 85%]
Lab6.py::test_substring_between_no_open PASSED                                            [ 88%]
Lab6.py::test_substring_between_no_close PASSED                                           [ 90%]
Lab6.py::test_substring_between_empty_open PASSED                                         [ 92%]
Lab6.py::test_substring_between_empty_close PASSED                                        [ 95%]
Lab6.py::test_substring_between_open_at_start PASSED                                      [ 97%]
Lab6.py::test_substring_between_close_at_end PASSED                                       [100%]
```

Вывод: В ходе работы я освоил приемы тестирования кода на примере использования пакета pytest.